

Tema 9

Errores y excepciones



Índice

- Introducción
- Excepciones comunes en C#
- Bloques try, catch y finally
- El objeto Exception
- throw manual de excepciones
- Propagación de excepciones
- Excepciones personalizadas
- Consejos y buenas prácticas trabajando con excepciones



- En todo desarrollo de software, los errores son **inevitables**.
- Estos pueden ocurrir debido a múltiples razones, como datos de entrada inesperados, errores de lógica, problemas de recursos, entre otros.
- En C# existen mecanismos para gestionar estos problemas de manera controlada, siendo las **excepciones** una herramienta clave.

1. Introducción



- En lo que llevamos de curso hemos visto distintas ocasiones en las que decíamos que "*saltaba una excepción*", como por ejemplo cuando accedíamos a un objeto `null` o cuando introducíamos por teclado texto en un tipo numérico.
- En este tema vamos a ver algunas de las excepciones más comunes, cómo tratarlas y cómo crear excepciones propias para intentar mitigar el efecto de los errores en tiempo de ejecución en nuestros programas.

1. Introducción

¿Qué es un error?

- Un **error** es cualquier condición anómala que hace que un programa se comporte de manera inesperada.
- Los errores pueden ser clasificados en dos grandes categorías:
 - Errores de tiempo de compilación
 - Errores de tiempo de ejecución

1. Introducción

- Los **errores en tiempo de compilación** se producen cuando el código fuente tiene problemas sintácticos o estructurales que impiden que el programa se compile.
- Ejemplos comunes son errores de sintaxis, tipos incorrectos o falta de referencias. Son errores fáciles de identificar y corregir, ya que se presentan al intentar compilar el programa.
- Al no poder compilar, estos errores no afectan al funcionamiento del programa.

1. Introducción

- Los **errores en tiempo de ejecución** ocurren mientras el programa se está ejecutando.
- Son más difíciles de predecir, ya que pueden depender de factores externos, como entradas del usuario o condiciones de red.
- Los errores de tiempo de ejecución, si no se gestionan adecuadamente, pueden hacer que **el programa falle inesperadamente**.

1. Introducción

¿Qué es una excepción?

- Una **excepción** es un tipo de error en tiempo de ejecución que puede ser detectado y manejado dentro del código.
- Cualquier excepción no controlada **provocará el cierre automático** del proceso que ejecuta el código.
- Es habitual que esto conlleve asociado el cierre completo del programa que se está ejecutando.

1. Introducción

- En C#, cuando ocurre una excepción, podemos **capturar y gestionar** este error en lugar de fallar abruptamente.
- Esto permite que el código reaccione ante errores de manera controlada, mejorando la estabilidad y experiencia del usuario.

1. Introducción

- El manejo de excepciones permite:
- **Controlar los errores de manera segura:** Los errores inesperados no terminan el programa de inmediato; en lugar de eso, son gestionados mediante bloques específicos.
- **Proveer una experiencia de usuario más robusta:** Al gestionar los errores, el software puede continuar ejecutándose o mostrar mensajes adecuados, en lugar de cerrarse de manera abrupta.

1. Introducción

- El manejo de excepciones permite:
- **Depurar el código más fácilmente:** La información de la excepción proporciona datos sobre el tipo y la ubicación del error, ayudando a los desarrolladores a identificar problemas más rápidamente.
- En aplicaciones "reales", un error no gestionado puede tener consecuencias graves, como pérdida de datos, interrupciones en el servicio o incluso riesgos de seguridad.

1. Introducción

- En C# existen numerosas excepciones predefinidas en la biblioteca de clases de .NET que cubren una amplia gama de errores.
- Conocer las excepciones más comunes y sus causas ayuda a gestionar y depurar problemas de manera efectiva.
- A continuación vamos a ver una lista de las excepciones más frecuentes.

2. Excepciones comunes en C#

NullReferenceException

- Posiblemente se trata de la excepción más común en C#, ya la hemos visto en algún tema con anterioridad.
- Esta excepción ocurre cuando el código intenta acceder a un miembro (como un método o propiedad) de un objeto que tiene un valor `null`.
- En otras palabras, se intenta utilizar un objeto que aún no ha sido inicializado.

2. Excepciones comunes en C#

- Una práctica común para evitarla es comprobar con un `if` si el objeto al que vamos a acceder es `null`.

- Ejemplo

2. Excepciones comunes en C#

```
1 Car bmw;  
2 bmw.Drive(500); // EXCEPCIÓN. el objet
```

```
1 Car bmw;  
2 if(bmw != null)  
3 {  
4     bmw.Drive(500); // No hay excepción, controlamos q  
5 }
```

IndexOutOfRangeException

- Ocurre cuando el código intenta acceder a un índice que está fuera de los límites de un arreglo o colección.
- Es importante asegurar que el índice está dentro de los límites válidos antes de acceder a él.

2. Excepciones comunes en C#

```
1 int[] numbers = { 14, -2, 83 }; ...
```

DivideByZeroException

- Esta excepción se lanza cuando el programa intenta dividir un número entre cero.
- Es común en operaciones matemáticas y puede prevenirse verificando que el divisor no sea cero.

2. Excepciones comunes en C#

```
1 int total = 10;  
2 int people = 0;  
3 Console.WriteLine(total / people); // Lanza DivideByZeroException
```


InvalidOperationException

- Esta excepción se lanza cuando una operación es inválida en el estado actual del objeto.
- Es común en escenarios en los que la lógica del programa intenta realizar una acción que no es válida en ese momento.
- Esta es la excepción que nos salta, por ejemplo, cuando intentamos obtener un objeto de una colección vacía.

2. Excepciones comunes en C#

Ejemplo

2. Excepciones comunes en C#

```
1 Queue<int> intQueue = new Queue<int>();  
2  
3 intQueue.Dequeue();  
4 // Lanza InvalidOperationException porque la cola
```

```
1 Stack<string> stringStack = new Stack  
2  
3 string.Pop();  
4 // Lanza InvalidOperationException
```

FormatException

- Ocurre cuando el formato de un valor es inválido y no puede ser convertido al tipo esperado.
- Es común en operaciones de conversión, como `int.Parse` o `double.Parse`.
- Es la excepción que nos saltaba cuando introducíamos texto como opción de un menú, por ejemplo.

2. Excepciones comunes en C#

```
1 string textString = "123a";  
2 int number = int.Parse(textString); // Lanza For
```

StackOverflowException

- Se produce cuando el programa alcanza el límite de llamadas en la pila de ejecución.
- Es común en situaciones de recursión infinita. A diferencia de otras excepciones, no se recomienda manejar esta excepción, ya que se trata de un error crítico que puede terminar el programa.
- Si creamos un algoritmo recursivo sin caso base, obtendremos esta excepción.

2. Excepciones comunes en C#

- Hasta ahora hemos visto qué son las excepciones y algunas de las más comunes, pero no sabemos cómo trabajar con ellas.
- En C#, los bloques `try`, `catch`, y `finally` permiten capturar y gestionar errores en tiempo de ejecución de manera controlada.
- Usarlos correctamente es fundamental para asegurar que los programas puedan manejar situaciones inesperadas sin detener su ejecución de forma abrupta.

3. Bloques `try`, `catch` y `finally`

Bloque try

- Un bloque try sirve para encapsular código que puede generar una excepción.
- Cuando el programa se encuentra dentro de un bloque try, evalúa si alguna de las instrucciones genera una excepción.
- Si una excepción ocurre, el flujo de ejecución salta al bloque catch correspondiente, si existe uno.

3. Bloques try, catch y finally

Bloque catch

- El bloque catch captura y maneja la excepción lanzada en el bloque try.
- Puede haber múltiples bloques catch para manejar diferentes tipos de excepciones de manera específica.
- Si no se especifica un tipo de excepción en catch, se capturará cualquier excepción, pero es recomendable capturar solo excepciones específicas cuando sea posible para evitar manejar errores que no se esperan.

3. Bloques try, catch y finally

Ejemplo

- En este ejemplo encapsulamos nuestro código en un bloque try. Si al ejecutar el código se produce una excepción de tipo `DivideByZeroException`, el código salta automáticamente al bloque catch.

3. Bloques try, catch y finally

```
1 try
2 {
3     int dividend = 0;
4     int result = 10 / dividend;
5 }
6 catch (DivideByZeroException ex)
7 {
```


8

```
Console.WriteLine("Error: No se puede dividir er
```

Ejemplo

- En este ejemplo tenemos 2 bloques catch diferentes para un mismo bloque try.

3. Bloques try, catch y finally

```
1  try
2  {
3      int[] numbers = { 1, 0, -32, 2 };
4      int result = 20 / numbers[1]; // Provoca una excepción de divi
5      Console.WriteLine(numbers[5]); // Provoca una excepción de ínc
6  }
7  catch (IndexOutOfRangeException ex)
```

```
8 {  
9     Console.WriteLine("Error: Índice fuera de rango.");  
10 }  
11 catch (DivideByZeroException ex)  
12 {  
13     Console.WriteLine("Error: No se puede dividir entre cero.");  
14 }
```

- Cada uno permite definir el código que vamos a ejecutar cuando se produzca una excepción del tipo que encapsula el catch.
- ¡OJO! Una vez que el código pasa a un bloque catch (ha saltado una excepción), **no se sigue ejecutando** el código del try.
- Si el método no es void, cada bloque catch debe tener una **expresión return válida** o elevar la excepción mediante el uso de throw, lo veremos más adelante.

3. Bloques try, catch y finally

- Si se produce una excepción de un tipo que no está previsto en un bloque catch, esta se quedará **sin capturar** (o manejar).
- Como hemos visto, esto provoca el cierre del proceso.
- Podemos capturar todas las excepciones mediante el uso genérico de `Exception` en el bloque catch.

3. Bloques try, catch y finally

Ejemplo

- En este ejemplo capturamos cualquier tipo de excepción, pero eso nos quita la capacidad de decidir qué hacer para cada una de ellas.
- Es preferible definir la(s) excepción(es) que queremos controlar que usar el bloque genérico.

3. Bloques try, catch y finally

```
1  try
2  {
3      int[] numbers = { 1, 0, -32, 2 };
4      int result = 20 / números[1]; // Provoca una excepción de divi
5      Console.WriteLine(numbers[5]); // Provoca una excepción de índ
6  }
```

```
7 catch (Exception ex)
8 {
9     Console.WriteLine("Error no especificado.");
10 }
```

Bloque finally

- El bloque `finally` es una parte opcional del manejo de excepciones en C#.
- A diferencia de `try` y `catch`, el bloque `finally` asegura que el código dentro de él se ejecutará independientemente de si una excepción fue lanzada o no.

3. Bloques `try`, `catch` y `finally`

- Su función principal es proporcionar una estructura segura para liberar recursos o ejecutar tareas de limpieza que deben llevarse a cabo sin importar el resultado del bloque try-catch.
- Se utiliza principalmente para liberar recursos externos, como ficheros o conexiones a bases de datos o para revertir cambios temporales en el estado de una aplicación.

3. Bloques try, catch y finally

- El bloque `finally` **siempre se ejecuta**, incluso si se llama a `return` dentro del bloque `try` o `catch`.
- Es importante destacar que el bloque `finally` se ejecutará **antes** de que el método retorne el control.
- Esta característica asegura que el `finally` se ejecute independientemente de otros flujos de control.

3. Bloques `try`, `catch` y `finally`

Ejemplo

3. Bloques try, catch y finally

```

1  static int Divide(int dividend, int divisor)
2  {
3      try
4      {
5          Console.WriteLine("Dividiendo {0}/{1}...", dividend, divisor);
6          return dividend / divisor;
7      }
8      catch (DivideByZeroException ex)
9      {
10         Console.WriteLine("ERROR no se puede dividir por 0");
11         return int.MaxValue;
12     }
13     finally
14     {
15         Console.WriteLine("Liberando recursos en finally");
16     }
17 }
18
19 // Llamada al método
20 Console.WriteLine("Resultado 10/2: " + Divide(10, 2));
21 Console.WriteLine();
22 Console.WriteLine("Resultado 10/0: " + Divide(10, 0));

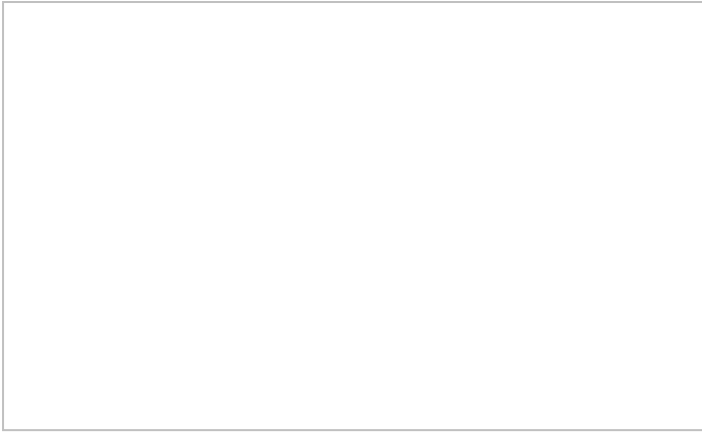
```

- En este ejemplo, tenemos un método que divide 2 enteros. Si el divisor es 0, salta una excepción que es capturada por el bloque `catch` correspondiente.
- Además, hay un bloque `finally` que se ejecuta siempre.
- Hacemos 2 llamadas al método, una que no genera excepción y otra que sí.

3. Bloques `try`, `catch` y `finally`

- El resultado es el siguiente:
- Vemos que el bloque finally se ejecuta en ambos casos.
- Además, se ejecuta **ANTES** de devolver el resultado y el control al método que hace la llamada.

3. Bloques try, catch y finally



Vídeo práctico

3. Bloques try, catch y finally

Vídeo práctico - código

3. Bloques try, catch y finally

```
1  static void Main(string[] args)
2  {
3      //Person person = null;
4      //Console.WriteLine(person.Name);
5
6      //int[] numbers = { 1, 2, 3, 4, 5 };
7      //Console.WriteLine(numbers[10]);
8
9      //int divisor = 0;
10     //Console.WriteLine(7 / divisor);
11
12     //Queue<int> number = new Queue<int>();
13     //number.Dequeue();
14
15     //int.Parse("1.5");
16
17     //Recursive(6);
18
19     try
20     {
21         Person person = new Person() { Name = "Al
```

- En C#, el objeto `Exception` es la **clase base** para todas las excepciones.
- Proporciona una **estructura común** para manejar y encapsular información de los errores que ocurren en tiempo de ejecución
- Esto permite que los desarrolladores controlen el flujo de los programas cuando se presentan situaciones inesperadas.

4. El objeto `Exception`

- La clase `Exception` se encuentra en el espacio de nombres `System`, y de ella **derivan** clases específicas de excepción como todas las que hemos visto en el punto 2.
- También podremos crear nuestros propios tipos de excepción mediante herencia de dicha clase.

4. El objeto `Exception`

Propiedades Principales de Exception

- La clase `Exception` contiene propiedades útiles que proporcionan detalles importantes acerca del error ocurrido. Las más importantes son las siguientes:
 - `Message`
 - `StackTrace`
 - `InnerException`
 - `Source`
 - `TargetSite`

4. El objeto `Exception`

Message

- Contiene una cadena que describe el error. Esta propiedad es útil para mostrar un mensaje informativo al usuario o al desarrollador.

4. El objeto Exception

```
1 try
2 {
3     int divisor = 0;
4     int result = 10 / divisor;
5 }
6 catch (DivideByZeroException ex)
7 {
8     Console.WriteLine(ex.Message);
9 }
```

Attempted to divide by zero.

StackTrace

- Proporciona una cadena que detalla la pila de llamadas en el momento en que se lanzó la excepción.
- Esto es muy útil para rastrear el origen exacto del error y depurar problemas en el código.
- En un programa complejo, la pila de llamadas contendrá numerosas entradas y esta propiedad nos resultará imprescindible para llegar al origen del error.

4. El objeto Exception

Ejemplo

4. El objeto Exception

```
1  static int Divide(int dividend, int divi
2  {
3      try
4      {
5          return 10 / divisor;
6      }
7      catch (DivideByZeroException ex)
8      {
9          Console.WriteLine(ex.StackTrace)
10         return 0;
11     }
12 }
13
14 static void Main(string[] args)
15 {
16     Console.WriteLine(Divide(10, 0));
17 }
```

InnerException

- Es otra instancia de `Exception` que contiene la excepción original que causó el error.
- Esta propiedad es útil cuando una excepción se lanza dentro de otra, lo que permite rastrear una cadena de errores hasta la excepción inicial.
- Lo veremos con más detalle en los 2 siguientes puntos de este tema.

4. El objeto `Exception`

Source

- Proporciona el nombre del programa o componente que lanzó la excepción, lo que permite identificar en qué parte de la aplicación ocurrió el error.

4. El objeto Exception

```
1 try
2 {
3     int divisor = 0;
4     int result = 10 / divisor;
5 }
6 catch (DivideByZeroException ex)
7 {
8     Console.WriteLine(ex.Source)
```

ConsoleApp

TargetSite

- Devuelve el método que lanzó la excepción. Esto es útil para identificar el lugar exacto en el código donde se generó el error.

4. El objeto Exception

```
1  static int Divide(int dividend, int
2  {
3      try
4      {
5          return dividend / divisor;
6      }
7      catch (DivideByZeroException ex)
8      {
9          Console.WriteLine(ex.TargetSite);
10         return 0;
11     }
12 }
13
14 static void Main(string[] args)
15 {
16     Console.WriteLine(Divide(10, 0))
17 }
```

Tnt32 Divide(Tnt32

Vídeo práctico

4. El objeto `Exception`

Vídeo práctico - código Person

4. El objeto Exception

```
1 public string Name { get; set; }
2 public int Age { get; set; }
3
4 public void GenerateException()
5 {
6     Console.WriteLine(1 / Age);
7 }
```

Vídeo práctico - código Program

4. El objeto Exception

```
1  static void Main(string[] args)
2  {
3      try
4      {
5          Person person = new Person() { Name = "Ant
6          person.GenerateException();
7          Console.WriteLine(7 / person.Age);
8          Console.WriteLine("Código que sigue a la €
9      }
10     catch (NullReferenceException ex)
11     {
12         Console.WriteLine(ex.Message);
13     }
14     catch (DivideByZeroException ex)
15     {
16         Console.WriteLine(ex.TargetSite);
17     }
18     finally
19     {
20         Console.WriteLine("Liberamos recursos en 1
21     }
```

- El lanzamiento manual de excepciones en C# permite a los desarrolladores manejar proactivamente situaciones de error o condiciones anómalas en su código.
- En lugar de esperar a que una excepción ocurra "naturalmente", se puede usar la instrucción `throw` para **lanzar una excepción** de manera explícita cuando se detectan condiciones que requieran tratamiento especial o cuando los datos de entrada no cumplen ciertos requisitos.

5. throw manual de excepciones

Uso de throw

- La instrucción `throw` se utiliza para lanzar una excepción de forma intencional.
- Se puede lanzar cualquier clase que derive de la clase base `Exception`, incluyendo tanto excepciones estándar (como `ArgumentNullException`, `InvalidOperationException`) como excepciones personalizadas.

5. throw manual de excepciones

- Sintaxis:

- Ejemplo:

- En este ejemplo, se usa `throw` para lanzar una `ArgumentException` si la edad pasada como parámetro no cumple con el requisito de ser mayor de 18.

5. throw manual de excepciones

```
throw new ArgumentException("Mensaje de
```

```
1 public void VerifyAge(int age)  
2 {
```



```
3    if (age < 18)
4    {
5        throw new ArgumentException("La edad debe ser l
6    }
7
8    Console.WriteLine("Edad válida.");
```

Captura y relanzamiento de excepciones

- En algunos casos, es útil capturar una excepción y, luego, relanzarla para que sea manejada por otro bloque de código en una capa superior.
- Si estamos dentro de un bloque catch, podemos utilizar throw sin una nueva excepción para elevar directamente la excepción que ha llegado al catch.

5. throw manual de excepciones

- Esto se hace comúnmente en aplicaciones con múltiples capas (como servicios y bases de datos).
- Cada capa puede capturar la excepción, registrar el error y luego lanzarlo nuevamente para que el llamador (siguiente capa) lo gestione.

5. throw manual de excepciones

Ejemplo

- En este ejemplo, capturamos y procesamos la excepción, pero luego se relanza con `throw` para que la gestión final sea realizada por otra parte del programa.

5. `throw` manual de excepciones

```
1 public void ProcessData()  
2 {  
3     try  
4     {  
5         int resultado = 10 / 0; // Provoca una divis  
6     }  
7     catch (DivideByZeroException ex)
```

```
8      {  
9  
10     Console.WriteLine("Error en ProcessData: " +  
11     throw; // Relanza la misma excepción
```

- Mediante el uso de `throw` también podemos encapsular excepciones dentro de nuevas excepciones, mediante la propiedad `InnerException` que vimos anteriormente:

5. `throw` manual de excepciones

```
1  static int Divide(int dividend, int divisor)
2  {
3      try
4      {
5          return dividend / divisor;
6      }
7      catch (DivideByZeroException ex)
8      {
9          Console.WriteLine("Error en ProcessData: " + ex.Message);
10         throw new ArgumentException("Error, el argumento divisor no", ex);
11         // Lanza una nueva excepción con la anterior como InnerException
12     }
```

- En C#, cuando se lanza una excepción, el flujo del programa se interrumpe, y el control se transfiere al primer bloque catch que coincida con el tipo de la excepción.
- Este proceso de "escalar" la excepción a través de los métodos que la llamaron hasta encontrar un bloque catch adecuado se conoce como **propagación de excepciones**.

6. Propagación de excepciones

- La propagación de excepciones es fundamental en el manejo de errores en C# porque permite que las excepciones sean gestionadas en los niveles más adecuados del programa.

6. Propagación de excepciones

¿Cómo funciona?

- Cuando ocurre una excepción en un método, C# buscará un bloque catch en el mismo método que pueda manejar esa excepción.
- Si no encuentra un bloque catch adecuado en el método, la excepción se propaga al método llamador.

6. Propagación de excepciones

- Este proceso continúa hasta llegar al método principal (generalmente Main) o hasta que la excepción sea capturada por un bloque catch en algún nivel superior.
- Si no se encuentra ningún bloque catch que maneje la excepción, el programa terminará de manera abrupta, mostrando un mensaje de error que describe la excepción no manejada.

6. Propagación de excepciones

Ejemplo

6. Propagación de excepciones

```
1  public class Calculator
2  {
3      private int Divide(int divisor, int dividend)
4      {
5          if (dividend == 0)
6          {
7              throw new DivideByZeroException("No se puede
8          }
9          return divisor / dividend;
10     }
11
12     public void Calculate()
13     {
14         int resultado = Divide(10, 0); // Aquí se lanza
15         Console.WriteLine("Resultado: " + resultado);
16     }
17 }
18
19 public class Program
20 {
21     public static void Main()
22     {
```

En el ejemplo anterior:

1. En el método `Divide`, se lanza una excepción `DivideByZeroException` si el divisor es cero.
2. `Calculate` llama a `Divide` sin manejar la excepción, por lo que la excepción se propaga a `Calculate`.
3. Como `Calculate` tampoco tiene un bloque `catch`, la excepción continúa propagándose hasta `Main`.
4. Finalmente, en `Main`, el bloque `catch` captura la `DivideByZeroException` y muestra un mensaje de error.

6. Propagación de excepciones

- La propagación de excepciones permite una arquitectura de manejo de errores eficiente y organizada, ya que:
- **Reduce la complejidad:** Los métodos no tienen que manejar todas las excepciones posibles; pueden dejar que la excepción se propague y se maneje en un nivel superior.

6. Propagación de excepciones

- **Facilita el mantenimiento:** El código es más fácil de mantener, ya que el manejo de excepciones se puede centralizar en métodos de alto nivel, lo que evita la duplicación de bloques try-catch.
- **Permite encapsular lógica de manejo:** La propagación permite definir de manera clara qué nivel de la aplicación debe manejar qué tipo de errores.

6. Propagación de excepciones

Vídeo práctico

6. Propagación de excepciones

Vídeo práctico - código Person

6. Propagación de excepciones

```
1  public class Person
2  {
3      public string Name { get; set; }
4      private int _age;
5      public int Age
6      {
7          get { return _age; }
8          set
9          {
10             if (value < 0) throw new ArgumentException(
11                 _age = value;
12             }
13         }
14
15     public void GenerateException()
16     {
17         Console.WriteLine(1 / Age);
18     }
19 }
```

Vídeo práctico - código Program

6. Propagación de excepciones

```
1 public static void Main()  
2 {  
3     try  
4     {  
5         Person person = new Person() { Name = "Antonio"  
6         Exception(person);  
7         //Console.WriteLine(7 / person.Age);  
8         Console.WriteLine("Código que sigue a la excepc  
9     }  
10    catch (NullReferenceException ex)  
11    {  
12        Console.WriteLine(ex.Message);  
13    }  
14    catch (ArgumentException ex)  
15    {  
16        Console.WriteLine(ex.Message);  
17    }  
18    finally  
19    {  
20        Console.WriteLine("Liberamos recursos en finall  
21 }
```

- En C#, además de utilizar las excepciones estándar, es posible crear excepciones personalizadas que representen situaciones de error específicas de la lógica de negocio de la aplicación.
- Las excepciones personalizadas permiten una mayor claridad y especificidad en el código, haciendo que sea más sencillo para los desarrolladores identificar el origen y el contexto de un error.

7. Excepciones personalizadas

- Las excepciones personalizadas son útiles cuando las excepciones estándar (como `ArgumentException` o `FormatException`) no describen adecuadamente el problema que ha ocurrido.
- Al crear excepciones personalizadas, puedes proporcionar detalles específicos sobre el error, lo que facilita el mantenimiento, la depuración y la claridad del código.

7. Excepciones personalizadas

- Por ejemplo, si estamos programando la aplicación de un banco, podremos crear una excepción `InsufficientBalanceException` para representar un intento de retirada de dinero que supera el saldo disponible en la cuenta.
- Esto es más específico y fácil de entender que una excepción genérica.

7. Excepciones personalizadas

Cómo crear una excepción personalizada

- Para crear una excepción personalizada en C#, es necesario definir una nueva clase que herede de `Exception` o de alguna de sus subclases.
- Podemos añadir diferentes constructores con los parámetros que queramos, como un `string` para añadir información o una excepción para asignar el `InnerException`.
- También podemos añadir propiedades personalizadas que modelen mejor la excepción.

7. Excepciones personalizadas

Ejemplo

7. Excepciones personalizadas

```
1  public class InsufficientBalanceException : Exception
2  {
3      public InsufficientBalanceException()
4          : base("No se puede completar la transacción: saldo :
5      {
6      }
7
8      public InsufficientBalanceException(string message)
9          : base(message)
10     {
11     }
12
13     public InsufficientBalanceException(string message, Exce|
14         : base(message, innerException)
15     {
16     }
17
18     public InsufficientBalanceException(string message, Exce|
19         : base(message, innerException)
20     {
21         AttemptedWithdrawal = attemptedWithdrawal;
22     }
23 }
```

- En el ejemplo anterior creamos una excepción personalizada.
- Creamos varios constructores, uno sin parámetros y otros con parámetros para añadir información.
- También creamos una propiedad personalizada para añadir más información a la excepción.

7. Excepciones personalizadas

- Una vez tenemos nuestras propias excepciones, podemos lanzarlas con throw...

7. Excepciones personalizadas

```
1 public class BankAccount
2 {
3     public double Balance { get; private set; }
4
5     ...
6
7     public void Withdraw(double amount)
8     {
9         if (amount > Balance)
10        {
11            throw new InsufficientBalanceException("Error:
12
13        }
14        Balance -= amount;
15        Console.WriteLine("Retirada realizado con éxito. S
16    }
```

- ...y también podemos recogerlas con catch.

7. Excepciones personalizadas

```
1  BankAccount account = new BankAccount() { Balance
2
3  try
4  {
5      account.Withdraw(250);
6  }
7  catch (InsuficienteBalanceException ex)
8  {
9      Console.WriteLine(ex.Message);
10 }
```

Vídeo práctico

7. Excepciones personalizadas

Vídeo práctico - código OutOfStockException

7. Excepciones personalizadas

```
1  internal class OutOfStockException : Exception
2  {
3      public OutOfStockException(int quantity = 0)
4          : base("Intento de sacar más stock del restante")
5      {
6          RemovalQuantity = quantity;
7      }
8
9      public OutOfStockException(string message, int quantity)
10         : base(message)
11     {
12         RemovalQuantity = quantity;
13     }
14
15     public OutOfStockException(string message, Exception innerException)
16         : base(message, innerException)
17     {
18         RemovalQuantity = quantity;
19     }
20 }
```

Vídeo práctico - código Product

7. Excepciones personalizadas

```
1  internal class Product
2  {
3      public string Name { get; set; }
4      public int Stock { get; set; }
5
6      public void AddStock(int stock)
7      {
8          Stock += stock;
9          Console.WriteLine("Añadidas {0} unidades al stock ");
10     }
11
12     public void RemoveStock(int stock)
13     {
14         if (stock > Stock) throw new OutOfStockException("I
15
16         Stock -= stock;
17         Console.WriteLine("Retiradas {0} unidades del stock ");
18     }
19 }
```

Vídeo práctico - código Program

7. Excepciones personalizadas

```
1  static void Main(string[] args)
2  {
3      Product laptop = new Product() { Name = "Portatil", Stock =
4
5      laptop.AddStock(1);
6      laptop.RemoveStock(4);
7      try
8      {
9          laptop.RemoveStock(10);
10     }
11     catch (OutOfStockException ex)
12     {
13         Console.WriteLine(ex.Message + "\nIntento de sacar {0}
14     }
15 }
```

- El manejo de excepciones es esencial para crear aplicaciones robustas y confiables.
- Si bien las excepciones son útiles para capturar y tratar errores inesperados, el uso inadecuado de ellas puede llevar a problemas de rendimiento, código difícil de leer y depurar, o una experiencia negativa para el usuario final.
- A continuación, se detallan buenas prácticas y recomendaciones para manejar excepciones de forma efectiva en C#.

8. Consejos y buenas prácticas trabajando con excepciones

Uso de excepciones solo para errores

- Hay que evitar el uso de excepciones para controlar el flujo normal. Las excepciones están destinadas para situaciones inusuales o errores inesperados.
- Ejemplo de qué **NO** hacer.

8. Consejos y buenas prácticas trabajando con excepciones

```
1 try
2 {
3     if (person.Name == null)
4         throw new Exception("El nombre
5     }
6 catch (Exception ex)
7 {
8     Console.WriteLine(ex.Message);
9 }
```

Usar excepciones específicas

- En general, debemos evitar capturar excepciones genéricas (`catch(Exception ex)`).
- Las excepciones específicas ofrecen un mensaje claro sobre el tipo de error que ocurrió, facilitando la depuración y comprensión del código.
- Además, capturar todas las excepciones sin filtrar el tipo puede dificultar mucho la depuración cuando ocurren errores no relacionados con lo que esperamos.

8. Consejos y buenas prácticas trabajando con excepciones

Usar finally si utilizamos recursos

- Cuando trabajemos con recursos no manejados como archivos o conexiones a bases de datos, debemos utilizar el bloque finally para asegurarnos de que se liberan.
- Utilizando finally, nos aseguramos de su correcta gestión independientemente de si se produce una excepción como si no.

8. Consejos y buenas prácticas trabajando con excepciones

Capturar excepciones para gestionarlas

- Debemos evitar capturar excepciones si no vamos a gestionar el error.
- Poner un bloque try-catch y no hacer nada con las excepciones que se produzcan dificulta tanto la depuración de nuestro código como la correcta gestión de los errores que se produzcan.

8. Consejos y buenas prácticas trabajando con excepciones

Relanzar excepciones sin tratar

- Relacionado con el caso anterior, debemos evitar capturar y relanzar excepciones si no vamos a hacer ningún tratamiento con ellas.
- Si relanzamos una excepción, debe ser porque vamos a tratar la excepción capturada (por ejemplo, podemos guardar información) pero va a ser gestionada en una capa superior.

8. Consejos y buenas prácticas trabajando con excepciones

Excepciones personalizadas

- Se deben utilizar las excepciones personalizadas solo cuando aporten valor. Es decir, cuando la lógica no se representa bien con las excepciones estándar.
- Además, nuestras excepciones personalizadas deben tener un nombre que describa el error que representan y debemos implementar los constructores necesarios para proporcionar la información necesaria (mensaje, `InnerException` y otras propiedades personalizadas si las hay).

8. Consejos y buenas prácticas trabajando con excepciones

Código reducido dentro del try

- Debemos envolver en bloques try únicamente el código que realmente puede lanzar la excepción que queremos capturar.
- Si metemos código externo ocurre como cuando capturamos excepciones genéricas, podemos capturar errores que quedan fuera de nuestra intención, lo que provoca una mayor dificultad para depurar nuestros programas y tratar correctamente los errores que aparezcan.

8. Consejos y buenas prácticas trabajando con excepciones

Diapositiva final

En este tema hemos aprendido a trabajar con excepciones. Esto será fundamental si queremos desarrollar aplicaciones robustas y fiables.



Speaker notes